

Algorithms as Recipes

Visual review: complexity, sorting, and recurrences

with C++ mini-labs

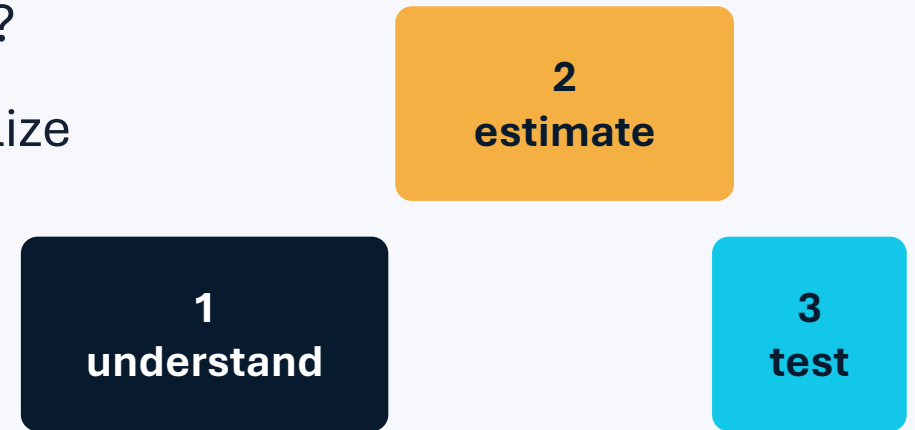
Fabrizio Ferrandi · Academic year 2023 - 2024

Advanced Algorithms and Parallel Programming - Review

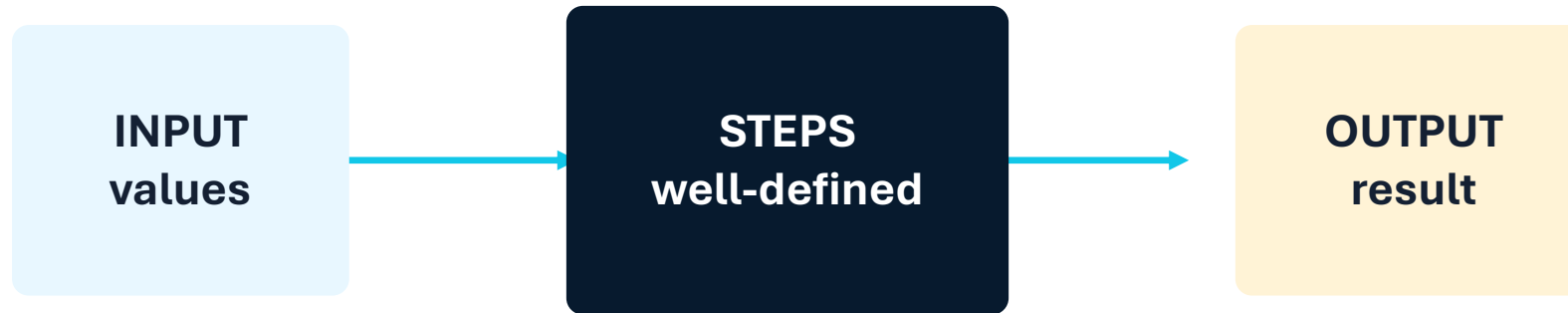


Make the key ideas quick to recall

- One idea per slide: less text, more structure.
- Guiding questions: what grows? where is the bottleneck?
- Colab examples: count operations, measure time, visualize curves.



An algorithm is a finite procedure



Remember

- It terminates in a finite number of steps
- It is described independently of the machine
- It can be compared with other algorithms

Larger input, longer time



$T(n)$ grows with n

more helpers $\neq$ always faster

the bottleneck remains

Guiding question: which part of the recipe can be parallelized?

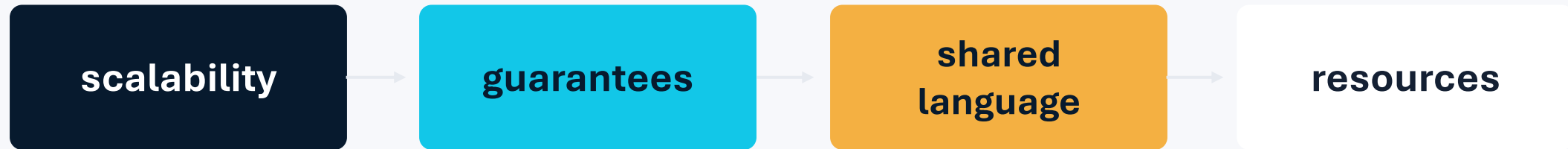


Shortest path

- Input: graph + source + destination
- Output: minimum-weight path
- Questions: does it exist? how much does it cost? can it be parallelized?

mini-lab: C++ / Dijkstra

Performance = the boundary between feasible and impossible



It is not the only criterion: correctness, maintainability, and simplicity still matter.

Analysis lets us reason without benchmarking every possible machine.

The problem of sorting

Input



sequence $\langle a_1, a_2, \dots, a_n \rangle$

Output



permutation $\langle a'_1, a'_2, \dots, a'_n \rangle$

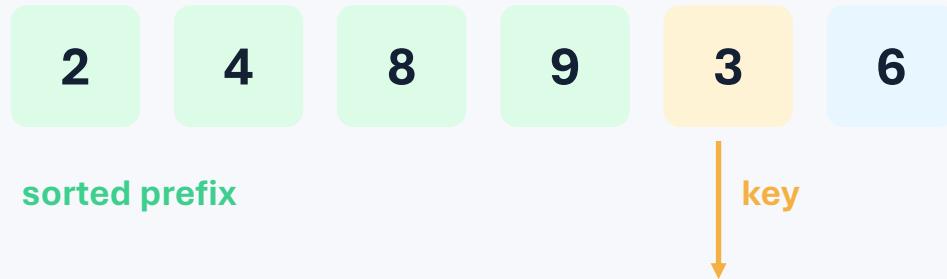
Correctness condition

- the output is ordered
- the output contains exactly the input values

Algorithmic question

- which procedure sorts it?
- how many operations does it need?
- how does the cost grow with n ?

Idea: keep a prefix sorted



```
for j = 2..n:  
  key = A[j]  
  shift right elements > key  
  insert key in the gap
```

- Great for small n or nearly sorted data
- Worst case: decreasing sequence
- Complexity: $\Theta(n^2)$ in the worst case

Measure insertion sort: comparisons and moves

Use the separate Colab notebook (C++) and change n / distribution.

```
pair<long long,long long> insertion_count(vector<int> A){
    long long comp = 0, shifts = 0;
    for (int j = 1; j < (int)A.size(); ++j){
        int key = A[j], i = j - 1;
        while (i >= 0 && (++comp, A[i] > key)){
            A[i+1] = A[i]; shifts++; i--;
        }
        A[i+1] = key;
    }
    return {comp, shifts};
}
```

Experiment

- sorted array
- random array
- reverse array

Expected result: the reverse case grows roughly like $n^2/2$.

Compile in Colab with g++ and vary n / input type.

Running time depends on the input

Best case
most favorable input

Average case
input from a
distribution

Worst case
guarantee for every
input

Algorithm analysis often uses the worst case: it is a robust promise.

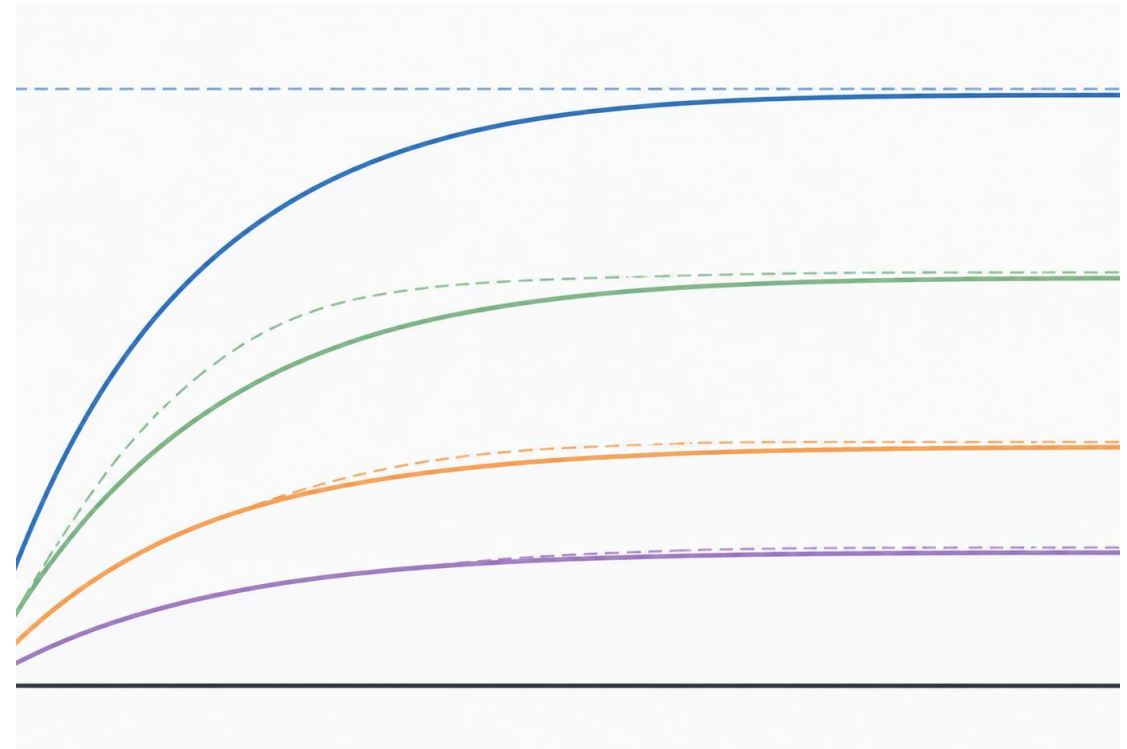
Look at growth, not machine details

$$3n^3 + 90n^2 - 5n + 6046$$

becomes

$$\Theta(n^3)$$

- Ignore multiplicative constants
- Drop lower-order terms
- Compare what happens as $n \rightarrow \infty$



Three symbols not to confuse

**$O(g(n))$
upper bound**

**$\Omega(g(n))$
lower bound**

**$\Theta(g(n))$
tight bound**

Rule of thumb

If $f(n)$ is both $O(g(n))$ and $\Omega(g(n))$, then $f(n)$ is $\Theta(g(n))$.

o and ω are strict comparisons

$$O / \Omega \approx \leq / \geq$$

$$o / \omega \approx < / >$$

$$2n^2 = o(n^3)$$

$$n^3 = \omega(n^2)$$

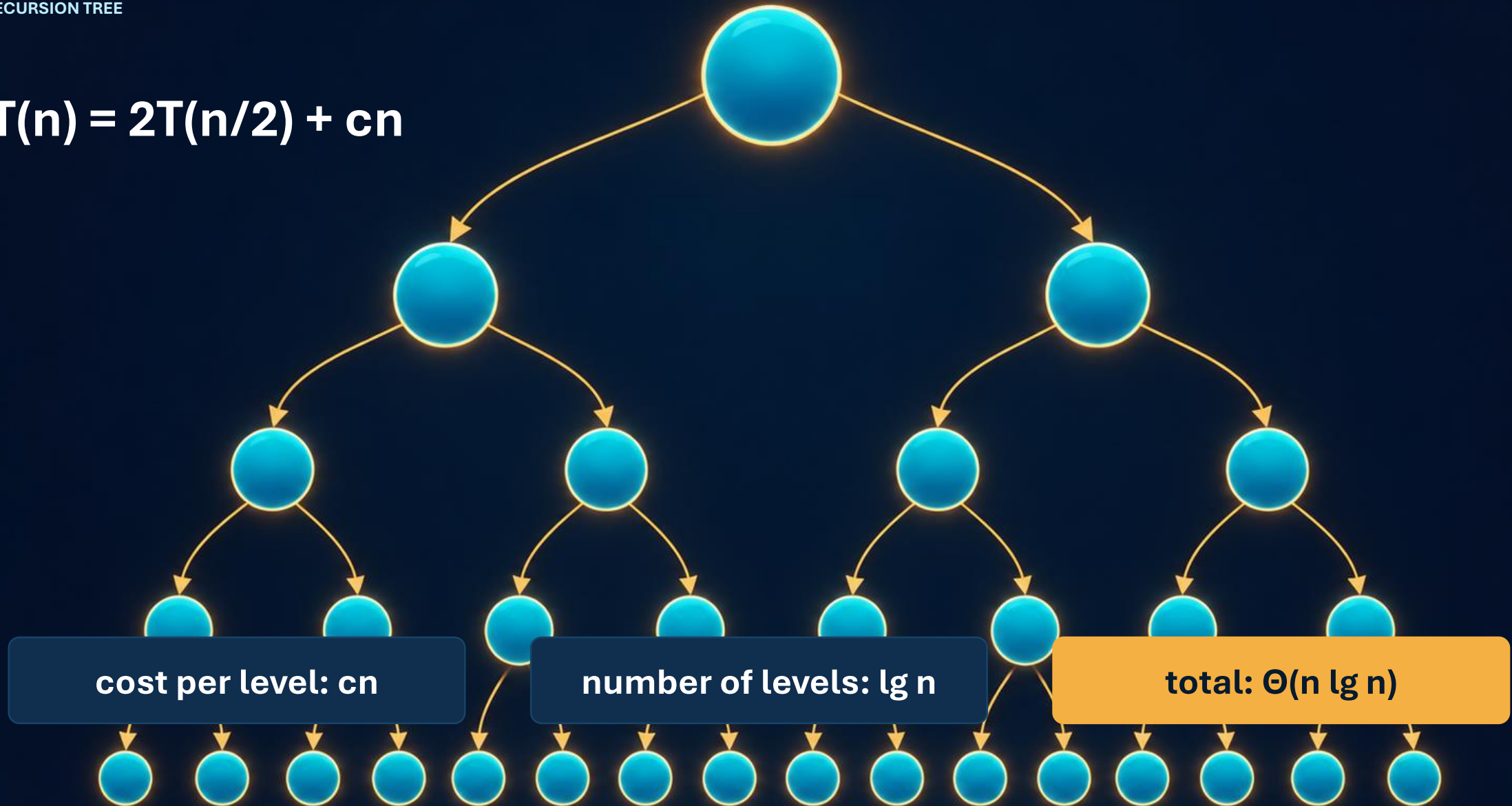
Divide, solve, combine



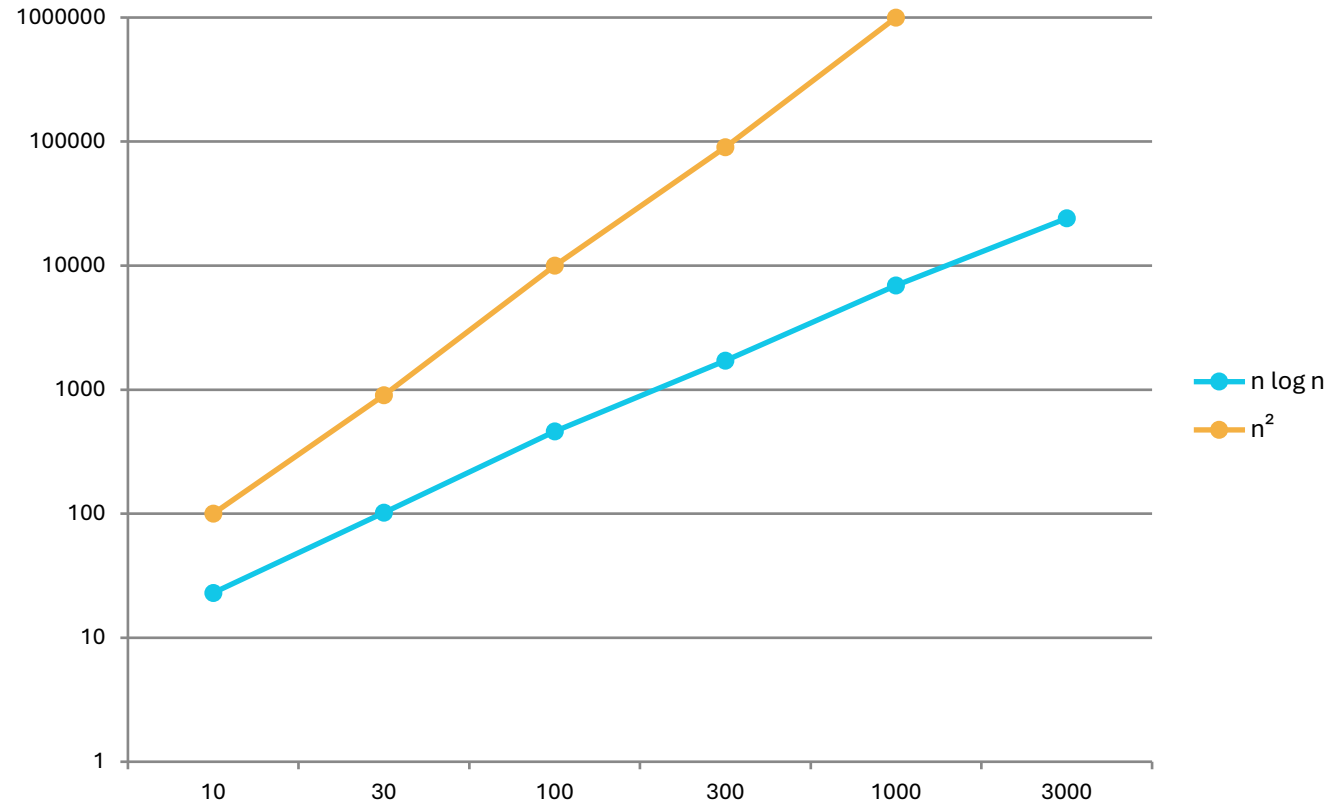
```
MERGE-SORT(A)
  if n == 1: return
  sort(left half)
  sort(right half)
  merge(sorted halves)
```

- Two subproblems of size $n/2$
- Merge cost: $\Theta(n)$
- Result: $\Theta(n \log n)$

$$T(n) = 2T(n/2) + cn$$



As n grows, $n \log n$ beats n^2



- Insertion sort: simple, fast for small inputs.
- Merge sort: more scalable for large inputs.
- In practice: test the crossover on real data.

Benchmark: insertion vs merge sort

Goal: see the crossover where the winner changes.

```
double bench(function<void(vector<int>&)> f, int n, int reps=5){
    vector<int> base(n); iota(base.begin(), base.end(), 0);
    double total = 0;
    for(int r=0; r<reps; r++){
        shuffle(base.begin(), base.end(), mt19937(r+7));
        auto A = base; auto t0 = chrono::high_resolution_clock::now();
        f(A); auto t1 = chrono::high_resolution_clock::now();
        total += chrono::duration<double, milli>(t1-t0).count();
    }
    return total / reps;
}
```



Discussion: why can hidden constants change the winner for small n ?

Colab flow: write file → g++ → run → compare timings.

Two practical ways to solve them



Substitution method

- general
- requires a good guess
- great for proofs

Master method

- fast
- only for $T(n) = aT(n/b) + f(n)$
- does not cover all cases

A recipe for many divide-and-conquer recurrences

$$T(n) = aT(n/b) + f(n)$$

$a \geq 1$
number of
subproblems

$b > 1$
shrink factor

$f(n)$
work outside
recursion

Step 1 Compute the reference term

$$n^{(\log_b a)}$$

Step 2 Compare $f(n)$ with that term

smaller | similar | larger

Step 3 Choose Case 1, 2, or 3

The theorem gives an asymptotic tight bound.

Three common cases

Compare $f(n)$ with $n^{\log_b a}$

Case 1
 $f(n)$ is polynomially smaller

$$f(n) = O(n^{\log_b a - \epsilon})$$

$$T(n) = \Theta(n^{\log_b a})$$

Case 2
 $f(n)$ has the same order

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

Case 3
 $f(n)$ is polynomially larger

$$f(n) = \Omega(n^{\log_b a + \epsilon})$$

regularity condition required

$$T(n) = \Theta(f(n))$$

Warning

The theorem is powerful, but it does not cover every recurrence. Boundary cases need another method.

Worked examples from the lecture

All examples use $T(n) = 4T(n/2) + f(n)$. Therefore $a = 4$, $b = 2$, and $n^{(\log_b a)} = n^2$.

$f(n) = n$
Case 1
 $T(n) = \Theta(n^2)$

$f(n) = n^2$
Case 2
 $T(n) = \Theta(n^2 \log n)$

$f(n) = n^3$
Case 3
 $T(n) = \Theta(n^3)$

$f(n) = n^2 / \log n$
No direct case
Master theorem does not apply

How to solve these quickly

- compute $n^{(\log_b a)}$ first
- compare only the dominant growth rate
- check regularity in Case 3
- do not force the theorem when no case matches

Recurrences and shortest paths in C++

```
long long T(long long n){
    if (n <= 1) return 1;
    return 2*T(n/2) + n;
}
for (int k = 1; k <= 10; ++k){
    long long n = 1LL << k;
    cout << n << " " << T(n) << " " << n*k << endl;
}
```

```
using State = pair<int,int>;
priority_queue<State, vector<State>, greater<State>>> pq;
dist[s] = 0; pq.push({0, s});
while (!pq.empty()){
    auto [d,u] = pq.top(); pq.pop();
    for (auto [v,w] : adj[u]) if (d + w < dist[v]){
        dist[v] = d + w; parent[v] = u;
        pq.push({dist[v], v});
    }
}
```

Questions for students

- Does $T(n)/(n \log_2 n)$ approach a constant?
- How does Dijkstra scale on sparse graphs?
- Which relaxations could run in parallel, and what still stays sequential?

In 5 sentences

Algorithm input $\rightarrow$ finite steps $\rightarrow$ output

Complexity studies how $T(n)$ grows

Notations O above, Ω below, Θ tight

Sorting insertion $\Theta(n^2)$, merge $\Theta(n \log n)$

Recurrences trees, substitution, Master Theorem

Final exercise: choose an algorithm, define $T(n)$, predict the curve, then verify in Colab.